MODUL PRAKTIKUM 06

ARSITEKTUR APLIKASI ANDROID

UI Layer • ViewModel • Repository • Data Layer • Separation of Concerns



Pemrograman Aplikasi Perangkat Bergerak (CIF62041)
S1 Teknik Informatika · Fakultas Ilmu Komputer · Universitas Brawijaya

Durasi Praktikum: 120 menit | Tahun 2026

1. Identitas dan Capaian Praktikum

Modul ini melanjutkan aplikasi NavigationLab dari Modul Praktikum 05. Fokus praktikum adalah merapikan struktur aplikasi agar tanggung jawab UI, state, akses data, dan sumber data tidak bercampur. Materi selaras dengan Materi 6 pada RPS: arsitektur aplikasi Android, UI layer, ViewModel, repository, data layer, model data, dan separation of concerns.

Komponen	Keterangan
Mata kuliah	Pemrograman Aplikasi Perangkat Bergerak
Materi	Arsitektur aplikasi Android
Sub-CPMK	Sub-CPMK103105 — Menerapkan navigasi dan arsitektur aplikasi Android berbasis pemisahan layer, ViewModel, repository, dan data layer pada aplikasi multi-screen.
Prasyarat	Project NavigationLab dari Modul Praktikum 05 dapat dijalankan tanpa crash.
Durasi	120 menit
Luaran	Source code, screenshot hasil, dan laporan singkat.

Capaian Praktikum

Setelah praktikum, mahasiswa mampu menerapkan pemisahan UI layer dan data layer, menggunakan ViewModel sebagai screen-level state holder, serta mengakses data melalui repository pada aplikasi Android berbasis Jetpack Compose.

Indikator Keberhasilan

- Composable screen tidak memuat data secara langsung dari data source.
- DetailViewModel memegang UI state untuk DetailScreen.
- Repository menjadi satu pintu akses data dari ViewModel.
- Data source berada pada data layer dan tidak diketahui langsung oleh UI.
- DetailScreen tetap menerima data dan callback sebagai parameter sehingga mudah diuji dan digunakan ulang.
- Aplikasi tetap dapat bernavigasi Home → Detail → Back tanpa crash setelah refactoring.

2. Alokasi Waktu 120 Menit

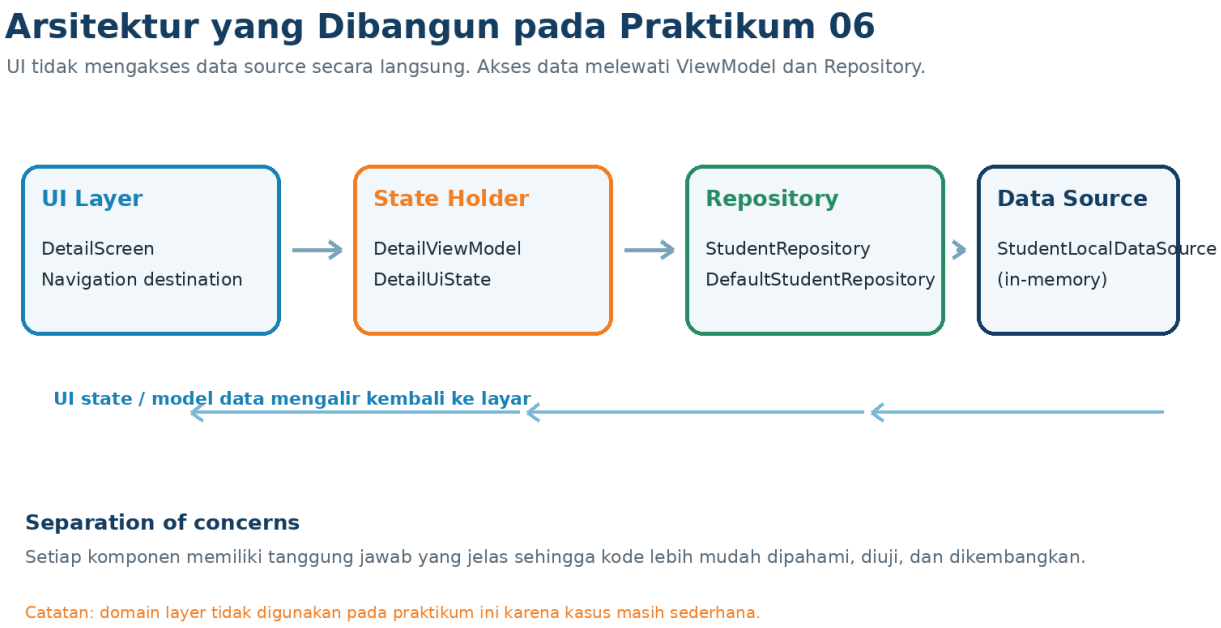
Tahap	Aktivitas	Durasi
1	Orientasi tujuan dan review project Modul 05	10 menit
2	Konsep singkat layered architecture & separation of concerns	20 menit
3	Membuat model, data source, dan repository	30 menit
4	Membuat UI state dan DetailViewModel	35 menit
5	Menghubungkan ViewModel ke Navigation dan DetailScreen	35 menit
6	Pengujian, debugging, dan screenshot	20 menit
7	Tugas pengembangan dan laporan singkat	20 menit

Prinsip Praktikum

Kita tidak menambahkan Room, REST API, Coroutines, atau Flow pada modul ini. Data masih in-memory agar fokus tetap pada struktur arsitektur. Teknologi tersebut akan masuk pada materi berikutnya.

3. Konsep Singkat Arsitektur Android

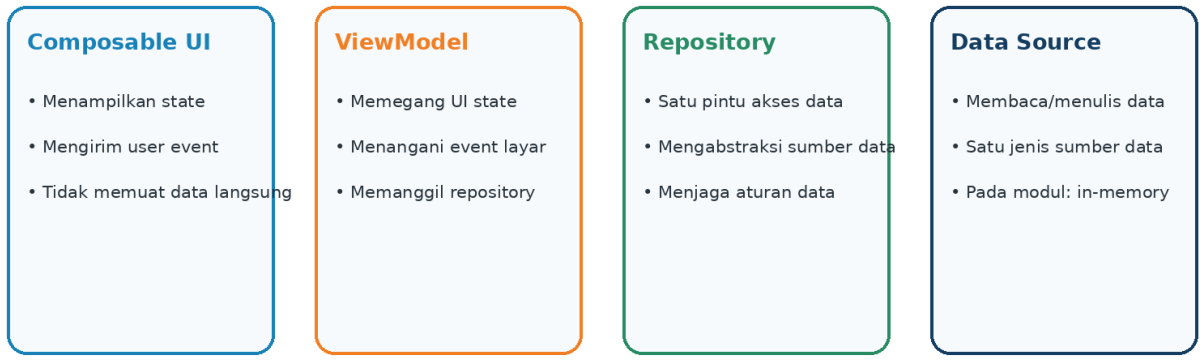
Panduan arsitektur Android modern merekomendasikan minimal dua lapisan: UI layer untuk menampilkan data dan menerima interaksi pengguna, serta data layer yang menyimpan aturan data dan mengekspos data melalui repository. Domain layer bersifat opsional dan tidak diperlukan pada kasus sederhana dalam modul ini.



Gambar 1. Arsitektur yang diterapkan pada NavigationLab.

Separation of Concerns

Separation of concerns berarti setiap bagian aplikasi memiliki tanggung jawab yang jelas. UI berfokus pada render dan user event, ViewModel memegang state tingkat layar dan mendelegasikan kebutuhan data, repository mengabstraksi akses data, sedangkan data source menangani sumber data tertentu.



Gambar 2. Pembagian tanggung jawab komponen.

Mengapa ViewModel?

ViewModel adalah state holder tingkat layar. Ia mengekspos state ke UI dan menjaga logika layar tetap berada di luar composable. ViewModel juga bertahan terhadap configuration change selama scope-nya masih hidup. Pada modul ini kita menggunakan state sederhana agar fokus pada arsitektur; pola StateFlow akan diperdalam ketika membahas Coroutines dan Flow.

4. Langkah Kerja

Langkah 1 — Buka Project NavigationLab dari Modul 05

1. Buka project NavigationLab hasil Modul Praktikum 05.
2. Jalankan aplikasi dan pastikan Home, Detail, Profile, dan About masih dapat dibuka.
3. Pastikan argument studentId pada Detail masih tampil dengan benar.
4. Buat commit/checkpoint awal sebelum refactoring jika menggunakan Git.

Checkpoint

Jangan lanjut jika project baseline masih error. Refactoring arsitektur harus dimulai dari aplikasi yang sudah dapat dijalankan.

Langkah 2 — Membuat Struktur Package

Tambahkan package berikut. Struktur tidak harus persis sama pada project production, tetapi pemisahan ini membantu melihat batas tanggung jawab setiap layer.

Struktur file yang disarankan

```
com.example.navigationlab/
├── data/
│   ├── model/
│   │   └── Student.kt
│   ├── source/
│   │   └── StudentLocalDataSource.kt
│   └── repository/
│       └── StudentRepository.kt
├── ui/
│   ├── detail/
│   │   ├── DetailUiState.kt
│   │   └── DetailViewModel.kt
│   └── screen/
│       └── DetailScreen.kt
├── navigation/
│   └── NavigationLabApp.kt
└── MainActivity.kt
```

Langkah 3 — Membuat Model Data Student

Model merepresentasikan data yang digunakan aplikasi. Pada tahap ini kita membuat model sederhana tanpa bergantung pada UI.

data/model/Student.kt

```
package com.example.navigationlab.data.model

data class Student(
    val id: Int,
    val name: String,
    val program: String,
    val year: Int
)
```

Perhatikan

Student tidak mengetahui DetailScreen, NavController, maupun ViewModel. Model data berada pada lapisan data dan dapat digunakan ulang oleh komponen lain.

Langkah 4 — Membuat Data Source In-Memory

Data source bertanggung jawab terhadap satu jenis sumber data. Untuk menjaga fokus praktikum, kita menggunakan list di memori sebagai sumber data sementara.

data/source/StudentLocalDataSource.kt

```
package com.example.navigationlab.data.source

import com.example.navigationlab.data.model.Student

class StudentLocalDataSource {
    private val students = listOf(
        Student(2026, "Alya Putri", "Teknik Informatika", 2026),
        Student(2027, "Bima Pratama", "Teknik Informatika", 2026),
        Student(2028, "Citra Lestari", "Teknik Informatika", 2026)
    )

    fun getStudentById(id: Int): Student? =
        students.find { it.id == id }
}
```

Langkah 5 — Membuat Repository

Repository menjadi satu pintu akses data bagi layer lain. ViewModel tidak perlu mengetahui apakah data berasal dari list, database, atau network.

data/repository/StudentRepository.kt

```
package com.example.navigationlab.data.repository

import com.example.navigationlab.data.model.Student
import com.example.navigationlab.data.source.StudentLocalDataSource

interface StudentRepository {
    fun getStudentById(id: Int): Student?
}

class DefaultStudentRepository(
    private val dataSource: StudentLocalDataSource
) : StudentRepository {
    override fun getStudentById(id: Int): Student? =
        dataSource.getStudentById(id)
}
```

Mengapa memakai interface?

UI/ViewModel bergantung pada kontrak StudentRepository, bukan implementasi konkret. Kelak implementasi dapat diganti menjadi Room atau REST tanpa mengubah DetailScreen.

Langkah 6 — Membuat UI State

UI state menyatakan data yang dibutuhkan layar untuk dirender. Gunakan satu objek state agar kondisi layar mudah dibaca.

ui/detail/DetailUiState.kt

```
package com.example.navigationlab.ui.detail

import com.example.navigationlab.data.model.Student

data class DetailUiState(
    val student: Student? = null,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)
```

Langkah 7 — Membuat DetailViewModel

ViewModel menerima repository melalui constructor. Ketika loadStudent() dipanggil, ViewModel meminta data ke repository lalu memperbarui uiState. Composable hanya membaca state tersebut.

ui/detail/DetailViewModel.kt

```
package com.example.navigationlab.ui.detail

import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.example.navigationlab.data.repository.StudentRepository

class DetailViewModel(
    private val repository: StudentRepository
) : ViewModel() {

    var uiState by mutableStateOf(DetailUiState())
        private set

    fun loadStudent(id: Int) {
        val student = repository.getStudentById(id)
        uiState = if (student != null) {
            DetailUiState(student = student)
        } else {
            DetailUiState(errorMessage = "Student tidak ditemukan")
        }
    }

    companion object {
        fun factory(repository: StudentRepository): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    return DetailViewModel(repository) as T
                }
            }
    }
}
```

Catatan tentang state

Contoh di atas sengaja menggunakan mutableStateOf agar konsep arsitektur tetap sederhana. Pada arsitektur Android modern, ViewModel umumnya mengekspos UI state sebagai StateFlow; hal tersebut akan dibahas saat masuk ke Coroutines dan Flow.

Langkah 8 — Refactor DetailScreen Menjadi UI Murni

DetailScreen tidak lagi menerima studentId mentah untuk mengambil data. Ia hanya menerima uiState dan callback. Dengan demikian screen tidak mengetahui repository atau data source.

ui/screen/DetailScreen.kt

```
@Composable
fun DetailScreen(
    uiState: DetailUiState,
    onBack: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("Detail Student", style = MaterialTheme.typography.headlineMedium)

        when {
            uiState.errorMessage != null ->
                Text(uiState.errorMessage)

            uiState.student != null -> {
                val student = uiState.student
                Text("ID: ${student.id}")
                Text("Nama: ${student.name}")
                Text("Program: ${student.program}")
                Text("Angkatan: ${student.year}")
            }
        }

        Button(onClick = onBack) {
            Text("Kembali")
        }
    }
}
```

Langkah 9 — Menyediakan Repository pada NavigationLabApp

Untuk praktikum ini dependency dibuat secara manual. Data source dan repository dibuat pada level aplikasi lalu repository diteruskan ke ViewModel. Pendekatan ini menunjukkan dependency direction tanpa menambahkan framework dependency injection.

Potongan NavigationLabApp.kt — dependency manual

```
val dataSource = remember { StudentLocalDataSource() }
val repository = remember { DefaultStudentRepository(dataSource) }

val navController = rememberNavController()

NavHost(
    navController = navController,
    startDestination = Routes.HOME
) {
    // destination lain tetap seperti Modul 05
}
```

Langkah 10 — Hubungkan DetailViewModel pada Destination Detail

ViewModel dibuat pada destination Detail. studentId tetap berasal dari argument Navigation, tetapi proses pengambilan data kini dilakukan oleh ViewModel melalui repository.

NavigationLabApp.kt — destination Detail

```
composable(
    route = Routes.DETAIL,
```

```
        arguments = listOf(
            navArgument("studentId") { type = NavType.IntType }
        )
    ) { backStackEntry ->
        val studentId = backStackEntry.arguments?.getInt("studentId") ?: -1

        val viewModel: DetailViewModel = viewModel(
            factory = DetailViewModel.factory(repository)
        )

        LaunchedEffect(studentId) {
            viewModel.loadStudent(studentId)
        }

        DetailScreen(
            uiState = viewModel.uiState,
            onBack = { navController.popBackStack() }
        )
    }
```

Arah Dependency

DetailScreen → DetailViewModel → StudentRepository → StudentLocalDataSource. Jangan membalik arah dengan membuat DetailScreen mengakses StudentLocalDataSource secara langsung.

5. Memahami Aliran Data dan Event

Langkah	Aliran
1	Home memanggil navigate("detail/2026")
2	NavHost membuka destination Detail dan membaca studentId
3	DetailViewModel.loadStudent(2026) dipanggil
4	ViewModel memanggil StudentRepository.getStudentById(2026)
5	Repository meneruskan permintaan ke StudentLocalDataSource
6	Model Student dikembalikan sampai menjadi DetailUiState
7	DetailScreen merender DetailUiState tanpa mengetahui sumber data

User Event Bergerak ke Dalam, State Bergerak ke Luar

Pola ini menjaga aliran program lebih mudah ditelusuri: event dari UI diteruskan ke state holder atau navigation, sedangkan data/state kembali ke UI untuk dirender. Pada tahap berikutnya pola yang sama akan diperluas menggunakan Flow dan sumber data nyata.

6. Pengujian Praktikum

Lakukan pengujian berikut setelah refactoring. Tandai setiap poin dan perbaiki error sebelum mengerjakan tugas akhir.

- ☐ Aplikasi dapat di-build dan dijalankan tanpa crash.
- ☐ Home tetap menjadi start destination.
- ☐ Tombol Detail membuka detail/2026.
- ☐ Detail menampilkan ID 2026, nama, program, dan angkatan dari repository.
- ☐ Tombol Kembali mengembalikan pengguna ke Home.

- ☐ Profile dan About dari Modul 05 tetap berfungsi.
- ☐ DetailScreen tidak mengimpor StudentLocalDataSource atau DefaultStudentRepository.
- ☐ DetailViewModel tidak mengimpor NavController atau composable UI.

Screenshot yang Wajib Diambil

- Screenshot 1 — struktur package data dan ui di Android Studio.
- Screenshot 2 — Detail screen menampilkan data Student ID 2026.
- Screenshot 3 — potongan DetailViewModel yang menggunakan StudentRepository.
- Screenshot 4 — potongan DefaultStudentRepository dan StudentLocalDataSource.
- Screenshot 5 — aplikasi kembali ke Home setelah Back.

7. Tugas Praktikum

Kembangkan fitur daftar mahasiswa pada Home tanpa melanggar batas layer yang sudah dibuat.

Ketentuan Implementasi

1. Tambahkan fungsi getStudents(): List<Student> pada StudentRepository dan implementasinya.
2. Tambahkan minimal dua data mahasiswa baru pada StudentLocalDataSource.
3. Buat HomeUiState dan HomeViewModel. HomeViewModel harus mengambil daftar Student melalui StudentRepository.
4. Ubah HomeScreen agar menerima List<Student> atau HomeUiState sebagai parameter dan menampilkan minimal ID + nama mahasiswa.
5. Ketika salah satu mahasiswa dipilih, navigasikan ke Detail menggunakan ID mahasiswa tersebut.
6. HomeScreen tidak boleh mengakses StudentLocalDataSource atau DefaultStudentRepository secara langsung.
7. DetailScreen tetap bekerja menggunakan DetailViewModel yang sudah dibuat.
8. Pastikan tombol Back dan destination Profile/About tetap berjalan.

Challenge Opsional

Tambahkan state “student tidak ditemukan” yang lebih informatif, misalnya Card dengan tombol kembali ke Home. Jangan menambahkan Room atau REST API pada modul ini.

8. Luaran dan Format Laporan

A. Source Code

- Kumpulkan project Android Studio lengkap atau repository sesuai mekanisme kelas.
- Pastikan seluruh file data/model, data/source, data/repository, ui/detail, dan HomeViewModel ikut terkumpul.
- Project harus dapat di-build; jangan mengumpulkan hanya potongan kode atau screenshot.

B. Screenshot

Sertakan lima screenshot wajib dari bagian pengujian dan minimal satu screenshot hasil tugas yang menampilkan daftar mahasiswa pada Home.

C. Laporan Singkat

Laporan cukup 2–3 halaman di luar cover dengan struktur berikut:

Bagian	Isi
1. Identitas	Nama, NIM, kelas, modul praktikum.
2. Tujuan	Tujuan praktikum dalam 2–3 kalimat.
3. Struktur Arsitektur	Jelaskan peran UI, ViewModel, repository, dan data source pada project Anda.
4. Hasil	Screenshot utama dan penjelasan singkat aliran data.
5. Tugas	Jelaskan perubahan untuk daftar mahasiswa pada Home.
6. Kendala & Solusi	Error/kendala yang ditemui dan bagaimana Anda memperbaikinya.
7. Kesimpulan	Apa manfaat separation of concerns yang Anda rasakan dari refactoring ini?

9. Troubleshooting Singkat

Masalah	Penyebab yang Sering Terjadi	Perbaikan
Unresolved reference viewModel	Dependency lifecycle-viewmodel-compose belum tersedia atau import salah.	Gunakan import androidx.lifecycle.viewmodel.compose.viewModel dan pastikan dependency ViewModel Compose tersedia pada project.
Detail selalu “tidak ditemukan”	studentId yang dikirim tidak ada pada data source.	Samakan ID pada tombol Home dengan ID di StudentLocalDataSource.
ViewModel dibuat berulang	ViewModel dibuat di luar scope destination atau factory berubah secara tidak stabil.	Buat ViewModel pada destination menggunakan viewModel(factory = ...).
UI tidak berubah	uiState tidak menggunakan observable Compose state atau tidak diperbarui.	Pastikan uiState menggunakan mutableStateOf dan setter dipanggil dari loadStudent().
Screen masih mengakses repository	Refactoring belum memisahkan UI dari data layer.	Pindahkan pemanggilan repository ke ViewModel dan kirim state ke screen.
Import Student ambigu	File model berada di package berbeda atau ada class Student duplikat.	Periksa package dan hapus model duplikat.

10. Ringkasan

Pada praktikum ini NavigationLab direfactor menjadi aplikasi berlapis. UI hanya merender state dan mengirim event, ViewModel menjadi state holder tingkat layar, repository mengabstraksi akses data, dan data source menangani sumber data in-memory. Struktur tersebut mengurangi coupling dan mempersiapkan aplikasi untuk materi REST API, Coroutines/Flow, serta Room Database tanpa harus mengubah UI secara besar-besaran.

Ingat

Arsitektur bukan sekadar membuat banyak folder. Tujuan utamanya adalah membuat batas tanggung jawab yang jelas dan arah dependency yang konsisten.

11. Referensi

[1] Android Developers. Guide to app architecture. https://developer.android.com/topic/architecture (diakses Oktober 2026).

[2] Android Developers. Recommendations for Android architecture. https://developer.android.com/topic/architecture/recommendations (diakses Oktober 2026).

[3] Android Developers. ViewModel overview. https://developer.android.com/topic/libraries/architecture/viewmodel (diakses Oktober 2026).

[4] Android Developers. Data layer. https://developer.android.com/topic/architecture/data-layer (diakses Oktober 2026).

[5] Ghita, C. (2022). Kickstart Modern Android Development with Jetpack and Kotlin. Packt Publishing.

[6] RPS Pemrograman Aplikasi Perangkat Bergerak (CIF62041), FILKOM Universitas Brawijaya, 2026.